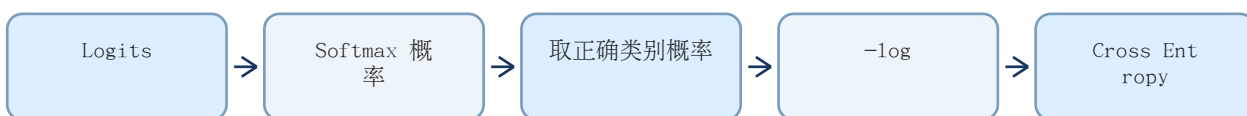


第18讲 实验设计

Cross Entropy、数值稳定性与 Next-token Loss

核心概念：实验设计文档

本讲不是让你背公式，而是让你能从 logits 亲手算出概率、正确类别概率、Cross Entropy，并能把这一过程重新放回 LLM 的 next-token prediction。



建议学习顺序：先看讲义 → 手算示例 → 运行实验代码 → 做练习 → 用速查卡复盘。

一、实验目标与研究问题

本实验不是做性能竞赛，而是验证一组可以直接支撑后续 Gradient / LLM 训练理解的机制性结论。

RQ	研究问题	对应实验
RQ1	Cross Entropy 是否只由真实类别概率决定？	实验1/2/4
RQ2	提高正确类别 logit 是否单调降低 CE？	实验3
RQ3	Batch mean CE 怎样由每个样本 loss 聚合？	实验5
RQ4	naive exp/softmax 在大 logits 下是否数值不稳定？	实验6
RQ5	NumPy 手写结果是否与 PyTorch 一致？	实验7
RQ6	LLM next-token CE 如何从 (T,V) / (B,T,V) 得到？	实验8/9

二、可检验假设

- H1: 同一组概率下，target 指向哪个类别，CE 就等于那个类别概率的负对数。
- H2: 其他 logits 固定时，提高正确类别 logit 会提高 p_correct，并降低 CE。
- H3: one-hot 完整公式与类别 index 形式得到完全相同的 CE。
- H4: 大 logits 下 naive exp 会溢出，而 max-shift 稳定 Softmax 保持有限。
- H5: NumPy 稳定实现与 PyTorch F.cross_entropy 的结果在浮点误差范围内一致。
- H6: (B,T,V) 的 causal LM logits 可以通过 flatten 为 (B*T,V) 后，对每个 token 位置统一计算 CE。

三、变量与控制

类别	变量	控制方式
自变量	target index、正确类 logit、大数 logits、Batch/Token Shape	只改一个因素，其余保持固定
因变量	p_correct、per-sample CE、mean CE、是否出现 inf/nan	统一打印到 TXT
控制变量	其他类别 logits、公式实现、dtype	NumPy float64 用于精确对照
外部核对	PyTorch F.cross_entropy	同一批 logits / target

科研连接：为什么这也算“实验设计”？

科研实验的最小骨架就是：明确问题 -> 提出可检验假设 -> 控制变量 -> 运行可复现代码 -> 记录结果 -> 判断假设。即使实验很小，这个结构也值得从现在开始固定。

四、实验环境

项目	要求
Python	3.x
NumPy	必需
Matplotlib	必需，用于三张结果图
PyTorch	用于对照；若无则可跳过实验7
GPU	不需要，CPU 足够
随机性	本实验无随机采样，因此无需随机种子

五、实验步骤

步骤1: 运行单样本三分类, 记录 logits、Softmax、p_correct、CE。

步骤2: 保持 logits 不变, 轮流改变 target index, 观察 loss 怎样变化。

步骤3: 只改变正确类别 logit, 从 -2 增加到 5, 记录 p_correct 和 CE, 并画趋势图。

步骤4: 用 one-hot 完整公式与 $-\log(p_{\text{correct}})$ 计算同一 loss, 比较差值。

步骤5: 构造 3x4 Batch logits, 计算 per-sample loss 和 mean loss。

步骤6: 输入 [1000,1001,999], 比较 naive 和 stable 实现。

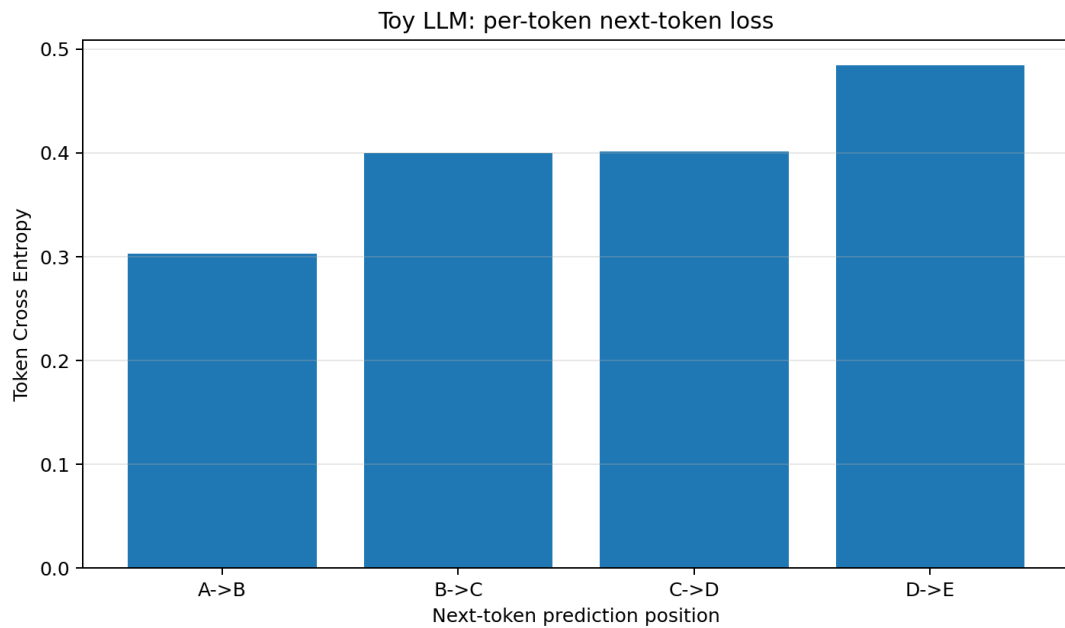
步骤7: 用 PyTorch F.cross_entropy 对照 NumPy; 再故意先 softmax, 展示常见错误。

步骤8: 构造 A->B->C->D->E 玩具序列, 计算 4 个 next-token loss。

步骤9: 构造 (B,T,V) logits, flatten 成 (B*T,V), 检查 Shape 与平均 token loss。

六、预期结果与验收标准

检查项	预期结果	通过标准
单样本 CE	$[2,1,0.1]$, target=0 $\rightarrow$ 0.417030	误差 $< 1e-6$
one-hot 等价	两种计算差值为 0	浮点误差内相等
Batch mean	约 0.300668	与手写结果一致
稳定性	naive 出现 inf/nan; stable 有限	能解释原因
PyTorch 对照	max abs diff $\sim 5.55e-17$	$< 1e-10$
LLM toy loss	4 个位置分别有 token loss	能解释 target shift
Shape	$(B,T,V) \rightarrow (B*T,V)$	V 始终保留为类别轴



玩具 LLM 中，每个 next-token 位置都有自己的 Cross Entropy；最后再聚合。

七、实际结果摘要

实验	实际结果
单样本 target=0	p=0.659001, CE=0.417030
同 logits target=2	p=0.098566, CE=2.317030
Batch	per-sample=[0.185182,0.263733,0.453090], mean=0.300668
大 logits naive	exp=[inf,inf,inf], probability=[nan,nan,nan]
大 logits stable	prob=[0.244728,0.665241,0.090031], CE=0.407606
PyTorch 对照	最大绝对差 5.55e-17
错误 API 用法	softmax 后再 CE 得 0.938085, 而正确为 0.300668
Toy LLM	平均 next-token CE=0.397262

八、结果解释

Cross Entropy 不是在比较“预测类别编号与真实编号差多少”，而是在惩罚真实类别拿到的概率过低。

提高正确类 logit 会通过 Softmax 的相对竞争提高 p_correct, 因此 CE 下降。

框架直接接 logits 有数学与数值稳定两层原因；不要因为概率更“直观”就先 softmax。

LLM 的 language modeling loss 可以理解成大量 token-level 分类 loss 的平均，而不是一句话只有一个 loss。

九、复现实验与故障排查

```
cd <解压后的第18讲目录>
python lesson18_cross_entropy_experiment.py
```

正常情况下，终端会打印 9 个实验区块，并在 `figures/` 生成 3 张 PNG。

现象	可能原因	处理
ModuleNotFoundError: numpy	未安装 NumPy	pip install numpy
ModuleNotFoundError: matplotlib	未安装绘图库	pip install matplotlib
没有 PyTorch	实验7跳过即可	后续 PyTorch 阶段再安装
出现 nan	检查是否运行 naive 大 logits 实验	这是预期现象; stable 结果应有限
CE 数值不同	target index 或 logits 被改动	恢复原始常量再对照 TXT

科研连接：实验记录要求

不要只看“代码能跑”。至少记录：输入 Shape、target、关键中间概率、per-sample loss、mean loss、异常输出和解释。后续做科研实验也沿用这种“结果 + 解释 + 可复现条件”的记录习惯。